JAVA PROGRAM REPORT

Mobile Number Validation Using Exception Handling

1. Objective / Aim

To validate a mobile number entered by the user and demonstrate built-in and user-defined exception handling based on its length and characters.

2. System Requirements

Hardware: Standard PC/laptop, keyboard, monitor, and sufficient memory and storage.

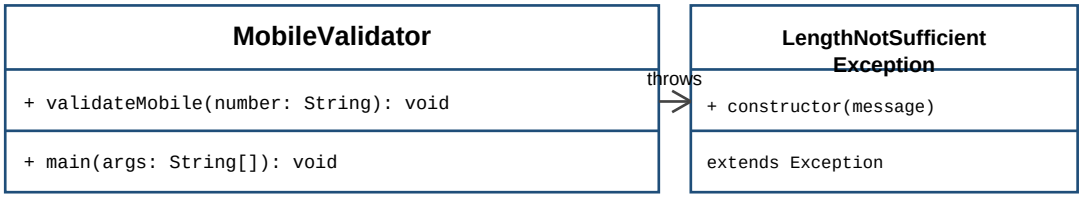
Software: Windows/Linux/macOS, JDK, and a Java IDE or command prompt/terminal.

3. Algorithm / Logic Description

1. Start and prompt the user to enter a mobile number.
2. Read the input as a String using Scanner.
3. Check every character; if any character is not a digit, throw NumberFormatException.
4. If the number contains more than 10 characters, throw ArrayIndexOutOfBoundsException.
5. If the number contains fewer than 10 characters, throw LengthNotSufficientException.
6. If all checks pass, print “Valid number”.
7. Catch each exception and display its corresponding message.
8. Close Scanner in the finally block and stop.

4. UML Class Diagram

The diagram shows MobileValidator and the custom exception class.



5. Source Code

```
import java.util.Scanner;

// User-defined exception
class LengthNotSufficientException extends Exception {
    public LengthNotSufficientException(String message) {
        super(message);
    }
}

public class MobileValidator {

    static void validateMobile(String number)
            throws LengthNotSufficientException {

        // Check whether all characters are digits
        for (int i = 0; i < number.length(); i++) {
            if (!Character.isDigit(number.charAt(i))) {
                throw new NumberFormatException();
            }
        }

        // Check whether the number exceeds 10 digits
        if (number.length() > 10) {
            throw new ArrayIndexOutOfBoundsException();
        }
        // Check whether the number has fewer than 10 digits
```

```
        if (number.length() < 10) {
            throw new LengthNotSufficientException(
                    "Invalid Mobile Number - LengthNotSufficientException");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter mobile number: ");
        String number = sc.nextLine();

        try {
            validateMobile(number);
            System.out.println("Valid number");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(
                    "Invalid Mobile Number-ArrayIndexOutOfBounds Exception");
        } catch (LengthNotSufficientException e) {
            System.out.println(e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println(
                    "Invalid Mobile Number -NumberFormatException");
        } finally {
            sc.close();
        }
    }
}
```

6. Compilation and Execution Commands

```
javac MobileValidator.java
java MobileValidator
```

7. Console Output Screenshot

Representative outputs for valid, too-short, too-long, and non-digit inputs:

```
C:\Java-Projects> javac MobileValidator.java
C:\Java-Projects> java MobileValidator
Enter mobile number: 1234567890
Valid number
C:\Java-Projects> java MobileValidator
Enter mobile number: 123456789
Invalid Mobile Number - LengthNotSufficientException
C:\Java-Projects> java MobileValidator
Enter mobile number: 12345678901
Invalid Mobile Number-ArrayIndexOutOfBounds Exception
C:\Java-Projects> java MobileValidator
Enter mobile number: 12ab567890
Invalid Mobile Number -NumberFormatException
```

8. Explanation

• **Scanner:** Reads the mobile number entered by the user.

• **NumberFormatException:** Thrown when a character is not a digit.

• **ArrayIndexOutOfBoundsException:** Used by this program to represent an input longer than 10 digits, though it is not the most semantically appropriate exception for this case.

• **LengthNotSufficientException:** A user-defined checked exception for inputs shorter than 10 digits.

• **try-catch-finally:** Handles exceptions and closes the Scanner.

Code check: The supplied code compiles and behaves as described for these test cases. For clearer exception design, an IllegalArgumentException or a custom TooLong exception would be more appropriate than ArrayIndexOutOfBoundsException for a number longer than 10 digits.